Solving the Celebrity Problem Using a Stack

Linked List Implementation from First Principles

Mahesh Kumar. P

Contents

1 Solving the Celebrity Problem Using a Stack

1.1 Introduction

Why?

Imagine entering a room and being asked to identify a famous guest. You cannot simply look at name tags. Instead, you may ask questions of the form: "Does person A know person B?" Each question costs time. The challenge is to find the special person with as few questions as possible.

The **Celebrity Problem** asks whether there is a person who is known by everyone else but who knows nobody else. This problem is famous in interviews because it tests more than coding syntax. It tests whether you can discover a hidden rule, eliminate impossible answers, prove your algorithm, and handle edge cases.

What?

A **celebrity** in this chapter is not necessarily a movie star. It is a person satisfying two strict conditions:

1. Everyone else knows the celebrity.
2. The celebrity knows no one else.

Why not try every pair of people? You could. But if there are n people, there are many possible relationships. A brute-force check may inspect about n^2 entries. When n is large, that becomes inefficient. Our goal is to use a simple but powerful elimination idea: one question can remove one person from consideration forever.

1.2 Real-World Analogies

1.2.1 Party Guests

At a party, everyone recognizes the guest of honor, but the guest of honor arrived quietly and knows nobody personally.

```
Alice ----> Celebrity
Bob   ----> Celebrity
Cara  ----> Celebrity
```

```
Celebrity ----> nobody
```

1.2.2 Office Meeting

A new CEO joins a company town hall. Every employee knows the CEO, but the CEO does not know individual employees yet.

1.2.3 Classroom

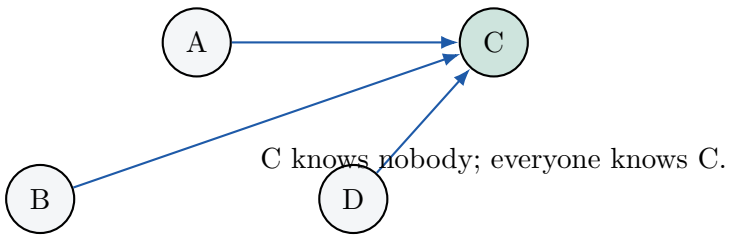
A famous professor visits a classroom. Every student knows the professor. The professor may not know any student by name.

1.2.4 Conference Attendees

At a research conference, all attendees recognize a keynote speaker. The keynote speaker may not know the attendees.

1.2.5 Social Media Followers

Think of "A knows B" as "A follows B." A celebrity is followed by everyone, but follows nobody in this group.



1.3 Problem Statement

Problem

Given n people numbered from 0 to $n-1$, and a relationship table M, where $M[i][j] = 1$ means person i knows person j, return the index of the celebrity if one exists. Return -1 if no celebrity exists.

Input. A square $n \times n$ matrix of zeros and ones. A matrix is a table with rows and columns. Row i, column j answers: "Does i know j?"

Output. One integer: the celebrity index, or -1.

Constraints.

- $n \geq 1$.
- The matrix must be square: every row has exactly n entries.

- Each entry is either 0 or 1.
- Usually $M[i][i] = 0$, because we do not need to ask whether someone knows themselves.

Example.

```
M = [
  [0, 1, 1],
  [0, 0, 1],
  [0, 0, 0]
]
Answer: 2
```

Person 2 knows nobody. Person 0 and person 1 both know person 2.

Non-example.

```
M = [
  [0, 1],
  [1, 0]
]
Answer: -1
```

Both people know someone, so neither can be a celebrity.

Edge cases. One person, no celebrity, everyone knows everyone, nobody knows anyone, malformed matrix, and misleading candidates that pass elimination but fail verification.

1.4 Understanding the Problem Visually

An arrow $A \to B$ means "A knows B." The arrow direction matters.

```
A ----> B means A knows B.
B ----> A means B knows A.
These are different facts.
```

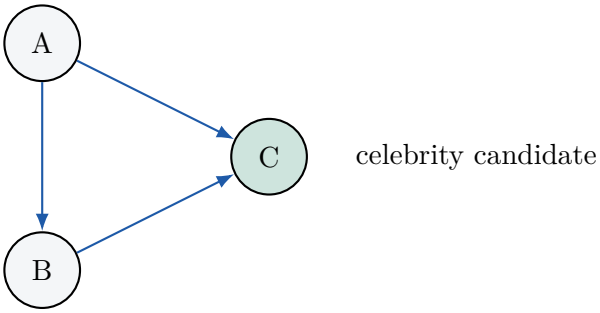
Example with C as celebrity:

```
A ----> B
A ----> C

B ----> C

C ----> nobody

Everyone knows C. C knows nobody.
```

Matrix view:

knows?	A	B	C
A	0	1	1
B	0	0	1
C	0	0	0

Graph view:



1.5 Mathematical Properties

Pause and Predict

Suppose A knows B. Which person is definitely not the celebrity? Think before reading the answer.

If A knows B, then A cannot be the celebrity, because a celebrity knows nobody. Person B might still be the celebrity.

Pause and Predict

Suppose A does not know B. Which person is definitely not the celebrity?

If A does *not* know B, then B cannot be the celebrity, because everyone must know the celebrity. Person A might still be the celebrity.

Mini Summary

One comparison knows(A, B) eliminates exactly one of the two people:

- If true, eliminate A.
- If false, eliminate B.

This is the heart of the optimized algorithm.

1.6 Naive Solution

Why?

Before optimizing, we need a baseline. A baseline is a simple solution that is easy to understand, even if it is slow.

Brute force means trying all possibilities directly. For each person c, we ask: "Could c be the celebrity?" That requires checking two things:

1. Row c: all zeros, because c knows nobody.
2. Column c: all ones except possibly $M[c][c]$, because everyone knows c.

```
FOR each candidate c from 0 to n-1:
    assume c is celebrity
```

```
    FOR each person p from 0 to n-1:
        IF p != c AND M[c][p] == 1: c is not celebrity
        IF p != c AND M[p][c] == 0: c is not celebrity
    IF c survived all checks: return c
return -1
```

Dry run intuition. Candidate 0 is checked against everyone. If candidate 0 fails, candidate 1 is checked from scratch. Work is repeated.

Complexity. There are n candidates. For each candidate, we may scan n people. Time is $O(n^2)$. Extra space is $O(1)$.

Limitation. The naive method does not use the elimination property. It forgets useful information learned from comparisons.

1.7 Optimization Journey

Can we eliminate people faster? Yes. If comparing A and B always removes one of them, then after comparing two people we never need to consider the eliminated person again.

Pause and Predict

If every comparison removes one candidate, how many comparisons are needed to reduce n people to one possible candidate?

Answer: $n - 1$ comparisons. Each comparison reduces the candidate pool by one.

This is like a tournament where each match removes one player. The final survivor is not automatically the winner, but is the only person who has not been disproven. We still verify them at the end.

1.8 Introduction to Stack

Why?

We need a simple structure that lets us repeatedly take two unresolved people, compare them, and put the survivor back. A stack fits this workflow beautifully.

A **stack** is a container where the last item inserted is the first item removed. This rule is called **LIFO**: Last In, First Out.

- **Push**: add an item to the top.
- **Pop**: remove and return the top item.
- **Peek**: look at the top item without removing it.
- **isEmpty**: check whether the stack has no items.

```
Push 0, push 1, push 2

Top
+---+
```

```
| 2 |  <- popped first
+---+
| 1 |
+---+
| 0 |
+---+
Bottom
```

1.9 Why Implement Stack Using Linked List?

A stack can be implemented with an array or a linked list.

Array stack. An array stores elements side by side. It is simple and cache-friendly, but may need resizing if capacity is fixed.

Linked-list stack. A linked list stores each element in a separate node. Each node stores data and a reference to the next node. It grows and shrinks naturally as operations happen.

Feature	Array Stack	Linked List Stack
Memory layout	Contiguous cells	Separate nodes connected by references
Growth	May need resizing	Dynamic by adding nodes
Push/pop top	Usually $O(1)$	$O(1)$
Extra memory	Low	Extra reference per node
Beginner value	Easy indexing	Excellent pointer practice
Use when	Capacity is known or resizing is fine	Size changes often or linked-list learning matters

1.10 Linked List Refresher

A **node** is a small object containing data and a reference to another node. A **reference** is a value that lets us find another object in memory. A **head** is the first node of a linked list.

```
Head
 |
 v
+------+     +------+     +--------+
| 10 | o---->| 20 | o---->| 30|null|
+------+     +------+     +--------+
```

To insert at the front:

1. Create a new node.
2. Point the new node to the old head.
3. Move head to the new node.

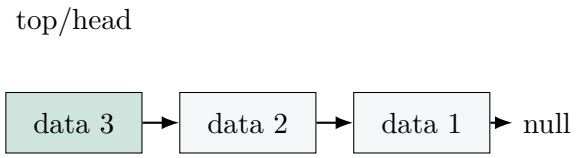
To delete from the front:

1. Save the old head's data.

2. Move head to the next node.
3. Return the saved data.

1.11 Stack Using Linked List

For a stack, the linked-list head is the stack top. This makes push and pop fast.



Push memory change.

```
Before push(4): top -> [3] -> [2] -> [1] -> null
After  push(4): top -> [4] -> [3] -> [2] -> [1] -> null
```

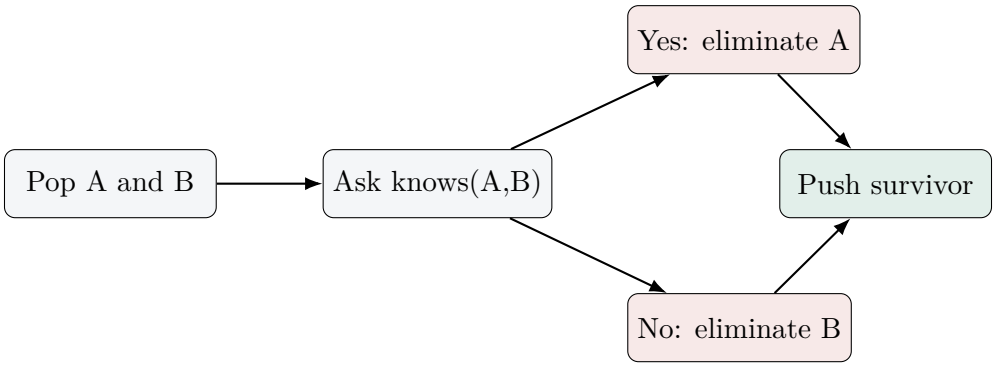
Pop memory change.

```
Before pop(): top -> [4] -> [3] -> [2] -> [1] -> null
After  pop(): top -> [3] -> [2] -> [1] -> null, returned 4
```

1.12 Why Stack Solves the Celebrity Problem

Push all people onto the stack. Then repeatedly pop two people. Compare them.

- If A knows B, A cannot be celebrity; push B back.
- If A does not know B, B cannot be celebrity; push A back.



Only one person survives because each comparison reduces stack size by one. But survival only means “not eliminated.” It does not prove celebrity status. Verification is required.

1.13 Complete Algorithm

1. Validate the matrix.
2. Create an empty linked-list stack.
3. Push every person index onto the stack.
4. While at least two people remain, pop two, compare, and push back the possible survivor.
5. Pop the final candidate.
6. Verify the candidate's row and column.
7. Return candidate if valid; otherwise return -1.

1.14 Proof of Correctness

Claim 1: Every eliminated person cannot be a celebrity. If A knows B, A violates the rule that a celebrity knows nobody. If A does not know B, B violates the rule that everyone knows the celebrity. Therefore each elimination is safe.

Claim 2: If a celebrity exists, they are never eliminated. When compared with any other person X, the celebrity knows X is false, so X is eliminated. If X is compared as A and celebrity as B, X knows celebrity is true, so X is eliminated. The celebrity survives.

Claim 3: The final survivor is the only possible celebrity. Since every other person was safely eliminated, no eliminated person can be the answer.

Claim 4: Verification is necessary. The final survivor may simply be the least-disproven person in a group with no celebrity. We must check that the survivor knows nobody and everyone knows the survivor.

Together, these claims prove the algorithm returns the celebrity exactly when one exists.

1.15 Language-Independent Pseudocode

```
FUNCTION findCelebrity(M):
    n = number of rows in M
    validate M is an n by n binary matrix

    stack = empty stack
    FOR person from 0 to n - 1:
        stack.push(person)

    WHILE stack.size() > 1:
        A = stack.pop()
        B = stack.pop()

        IF M[A][B] == 1:
            stack.push(B)       // A knows B, so A is not celebrity
        ELSE:
            stack.push(A)       // A does not know B, so B is not
               celebrity
```

```
    candidate = stack.pop()

    FOR person from 0 to n - 1:
        IF person == candidate:
            continue
        IF M[candidate][person] == 1:
            return -1
        IF M[person][candidate] == 0:
            return -1

    return candidate
```

Line explanation. The first loop prepares candidates. The while-loop performs elimination. The final loop verifies the two celebrity conditions.

1.16 Python Implementation

```python
class Node:
    """A single linked-list node used by the stack."""

    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedListStack:
    """Stack implemented using the head of a linked list as the top."""

    def __init__(self):
        self.top = None
        self._size = 0

    def push(self, value):
        new_node = Node(value)
        new_node.next = self.top
        self.top = new_node
        self._size += 1

    def pop(self):
        if self.is_empty():
            raise IndexError("Cannot pop from an empty stack")
        value = self.top.data
        self.top = self.top.next
        self._size -= 1
        return value

    def peek(self):
        if self.is_empty():
            raise IndexError("Cannot peek into an empty stack")
        return self.top.data

    def is_empty(self):
        return self.top is None

    def size(self):
        return self._size
```

```
40
41      def to_list(self):
42          values = []
43          current = self.top
44          while current is not None:
45              values.append(current.data)
46              current = current.next
47          return values
48
49
50  def validate_matrix(matrix):
51      if not matrix:
52          raise ValueError("Matrix must contain at least one person")
53
54      n = len(matrix)
55      for row in matrix:
56          if len(row) != n:
57              raise ValueError("Matrix must be square")
58          for value in row:
59              if value not in (0, 1):
60                  raise ValueError("Matrix entries must be 0 or 1")
61
62
63  def knows(matrix, person_a, person_b):
64      return matrix[person_a][person_b] == 1
65
66
67  def verify_candidate(matrix, candidate):
68      n = len(matrix)
69      for person in range(n):
70          if person == candidate:
71              continue
72
73          if knows(matrix, candidate, person):
74              return False
75
76          if not knows(matrix, person, candidate):
77              return False
78
79      return True
80
81
82  def find_celebrity(matrix):
83      validate_matrix(matrix)
84      n = len(matrix)
85
86      stack = LinkedListStack()
87      for person in range(n):
88          stack.push(person)
89
90      while stack.size() > 1:
91          person_a = stack.pop()
92          person_b = stack.pop()
93
94          if knows(matrix, person_a, person_b):
95              stack.push(person_b)
96          else:
97              stack.push(person_a)
```

```
98
99      candidate = stack.pop()
100
101     if verify_candidate(matrix, candidate):
102         return candidate
103     return -1
```

1.17 Driver Code

```
1  if __name__ == "__main__":
2      party = [
3          [0, 1, 1, 1],
4          [0, 0, 1, 0],
5          [0, 0, 0, 0],
6          [0, 1, 1, 0],
7      ]
8
9      result = find_celebrity(party)
10
11     if result == -1:
12         print("No celebrity found")
13     else:
14         print(f"Celebrity found at index: {result}")
```

1.18 Expected Console Output

```
Celebrity found at index: 2
```

The program prints this because person 2 knows nobody, and persons 0, 1, and 3 all know person 2.

1.19 Line-by-Line Code Walkthrough

Code element	Meaning
`class Node`	Defines the building block of the linked list.
`self.data`	Stores the person index inside the node.
`self.next`	Stores a reference to the next node.
`class LinkedListStack`	Defines stack behavior using linked nodes.
`self.top`	Points to the current top node.
`self._size`	Counts how many items are in the stack.
`push`	Creates a node and places it before the old top.
`pop`	Removes the top node and returns its data.
`peek`	Reads the top value without removing it.
`is_empty`	Returns whether the stack has no nodes.
`validate_matrix`	Rejects empty, non-square, or non-binary input.
`knows`	Gives a readable name to matrix lookup.
`verify_candidate`	Checks both celebrity rules.

Code element	Meaning
`find_celebrity`	Coordinates validation, elimination, and verification.
`return -1`	Reports that no valid celebrity exists.

1.20 Dry Run

Use this matrix:

```
0: [0, 1, 1, 1]
1: [0, 0, 1, 0]
2: [0, 0, 0, 0]
3: [0, 1, 1, 0]
```

After pushing 0, 1, 2, 3, the stack top is 3:

```
Top -> 3 -> 2 -> 1 -> 0 -> null
```

1. Pop 3 and 2. Does 3 know 2? Yes. Eliminate 3. Push 2.
2. Stack: `2 -> 1 -> 0`. Pop 2 and 1. Does 2 know 1? No. Eliminate 1. Push 2.
3. Stack: `2 -> 0`. Pop 2 and 0. Does 2 know 0? No. Eliminate 0. Push 2.
4. Stack: `2`. Candidate is 2.
5. Verify row 2: all zeros. Verify column 2: everyone else has 1. Return 2.

1.21 Trace Table

Iter.	Stack before	A	B	Comparison	Eliminated	Pushed	Candidate
1	3,2,1,0	3	2	$M[3][2] = 1$	3	2	unknown
2	2,1,0	2	1	$M[2][1] = 0$	1	2	unknown
3	2,0	2	0	$M[2][0] = 0$	0	2	2

1.22 Memory State

Before execution.

```
stack.top = null
size = 0
matrix stored separately as a table of rows
```

After pushes.

```
stack.top
   |
   v
 [3] -> [2] -> [1] -> [0] -> null
```

After first pop.

```
Returned 3
stack.top -> [2] -> [1] -> [0] -> null
```

After second pop.

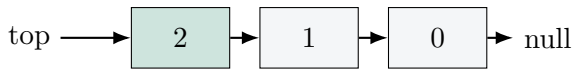
```
Returned 2
stack.top -> [1] -> [0] -> null
```

After pushing survivor 2.

```
stack.top -> [2] -> [1] -> [0] -> null
```

Final state.

```
Candidate = 2
stack is empty after candidate pop
verification reads matrix only
```



new survivor node

1.23 Complexity Analysis

Time complexity. Pushing all people takes $O(n)$. Elimination performs $n-1$ comparisons, so $O(n)$. Verification scans $n-1$ other people, so $O(n)$. Total time is $O(n)$.

Space complexity. The linked-list stack stores up to n person indices, so extra space is $O(n)$. The input matrix is not counted as extra space because it is given to the algorithm.

Worst case. $O(n)$, because we always eliminate down to one and verify.

Best case. Still $O(n)$ in this implementation, because we validate and verify. Some invalid verification may fail early, but asymptotically the safe bound is $O(n)$.

Average case. $O(n)$, because the number of elimination comparisons is fixed at $n-1$.

1.24 Edge Cases

Case	Expected behavior
Single person	Return 0 if self-entry is acceptable; one person vacuously knows nobody else and is known by everyone else.
No celebrity	Elimination returns a candidate, but verification returns -1.
Everyone knows everyone	Every person knows someone, so no celebrity.
Nobody knows anyone	For $n > 1$, nobody is known by everyone, so no celebrity.

Case	Expected behavior
Multiple apparent celebrities	Impossible under the strict definition for $n > 1$; verification prevents false positives.
Invalid matrix	Raise an error if empty, non-square, or containing values other than 0 and 1.

1.25 Common Beginner Mistakes

Common Trap

- Forgetting final verification and returning the stack survivor too early.
- Reversing the comparison: $M[A][B]$ is not the same as $M[B][A]$.
- Thinking the stack proves celebrity status. It only produces a candidate.
- Popping from an empty stack.
- In linked lists, forgetting to move `top` during `pop`.
- Losing the old top by assigning pointers in the wrong order during `push`.
- Off-by-one loops that skip person $n - 1$.

1.26 Debugging Techniques

- Print the stack after every push and pop using `to_list()`.
- Print each comparison as `Does A know B?`.
- Validate that the matrix is square before running the algorithm.
- Draw arrows for a small input by hand.
- Check row and column of the final candidate separately.
- For linked-list bugs, draw `top`, `next`, and `null` references.

1.27 Comparison with Other Approaches

Approach	Time	Extra Space	Readability	Interview suitability
Brute force	$O(n^2)$	$O(1)$	Very simple	Good baseline only
Stack	$O(n)$	$O(n)$	Clear elimination story	Excellent for stack topic
Two pointer	$O(n)$	$O(1)$	Compact but less visual	Excellent optimization
Candidate elimination variable	$O(n)$	$O(1)$	Minimal code	Best production variant

1.28 Interview Insights

Interviewers often ask this problem to see if you can communicate. A strong answer includes:

- State the celebrity definition precisely.
- Present brute force first, then optimize.
- Explain the two elimination rules.
- Emphasize that the survivor must be verified.
- Analyze time and space clearly.
- Mention the two-pointer $O(1)$-space variant as a follow-up.

Common follow-ups include: "Can you solve it without a stack?", "What if calls to knows are expensive?", "How do you validate input?", and "Can there be two celebrities?"

1.29 Industry Perspective

In production, the two-pointer or single-candidate approach is usually preferred because it uses $O(1)$ extra space. However, the stack approach is valuable when the system already has a stack abstraction, when teaching elimination, or when the candidate pool is naturally represented as pending work items.

Memory matters at scale. A linked-list stack allocates one object per person, which increases overhead. An array-based stack is often faster in real systems due to better locality. Maintainability also matters: if the stack version is clearer for a team, clarity may justify the small overhead for moderate n.

1.30 Frequently Asked Questions

1. **What is a celebrity?** A person known by everyone else who knows nobody else.
2. **What does $M[i][j] = 1$ mean?** Person i knows person j.
3. **Why is direction important?** Knowing is not necessarily mutual.
4. **Can two celebrities exist?** Not for $n > 1$, because each celebrity would have to know nobody while also knowing the other celebrity.
5. **Why use a stack?** It naturally stores unresolved candidates and supports pairwise elimination.
6. **Does the stack find the answer directly?** No. It finds a candidate.
7. **Why verify?** A group may have no celebrity, yet elimination still leaves one survivor.
8. **What is LIFO?** Last In, First Out: the newest pushed item is popped first.
9. **What is a linked list?** A chain of nodes connected by references.
10. **Why is push $O(1)$?** It changes only a few references at the head.
11. **Why is pop $O(1)$?** It removes only the head node.
12. **What if the matrix is empty?** The input is invalid.

13. **What if there is one person?** Return 0 under the usual definition.
14. **What if everyone knows everyone?** No celebrity exists.
15. **What if nobody knows anyone?** For more than one person, no celebrity exists.
16. **Is this a graph problem?** Yes, it can be viewed as a directed graph problem.
17. **What is a directed graph?** A set of nodes with arrows that have direction.
18. **What is time complexity?** A way to describe how running time grows with input size.
19. **What is space complexity?** A way to describe how extra memory grows with input size.
20. **Is stack optimal?** Time is optimal at $O(n)$, but space can be improved to $O(1)$.
21. **Can I use Python's list as a stack?** Yes, but this chapter implements a linked-list stack for learning.
22. **What is the biggest mistake?** Returning the survivor without verification.

1.31 Expert Tips

Expert Tip

- Memorize the elimination rule as: "If A points to B, A is out; if A does not point to B, B is out."
- Say "candidate" until verification passes. Only then say "celebrity."
- In diagrams, draw arrows into the celebrity and no arrows out.
- In code, wrap matrix access in a `knows` function to avoid direction mistakes.
- For interviews, first explain brute force in one minute, then improve it.

1.32 Summary

The Celebrity Problem asks for a person known by everyone else and knowing nobody else. The key observation is that one comparison eliminates one person. A stack stores unresolved people, lets us pop two at a time, and pushes back the only possible survivor. After $n-1$ eliminations, one candidate remains. Verification checks the candidate's row and column. The stack solution runs in $O(n)$ time and uses $O(n)$ extra space.

1.33 Cheat Sheet

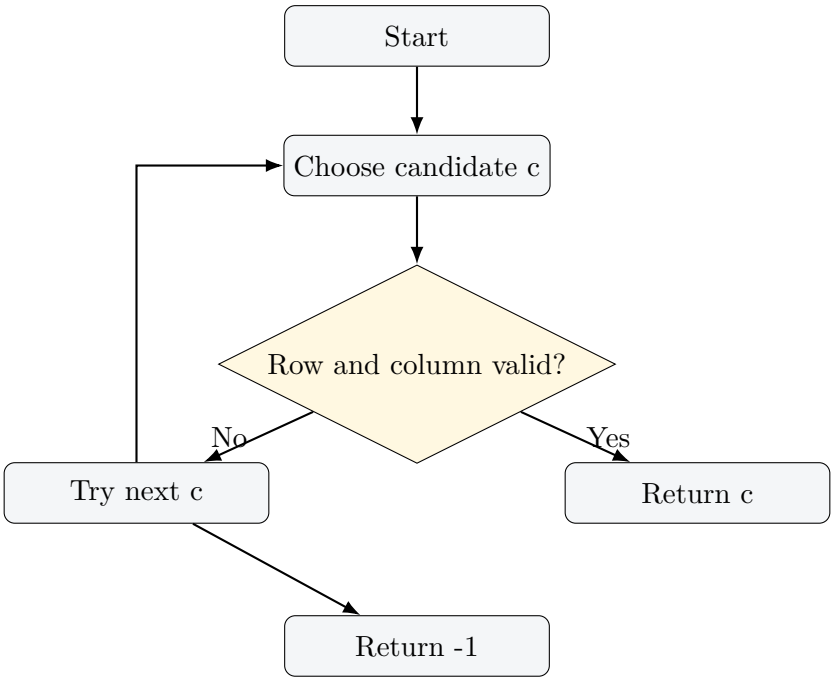
Item	Reminder
Celebrity	Known by everyone, knows nobody
Matrix entry	$M[i][j] = 1$ means i knows j
If A knows B	A cannot be celebrity
If A does not know B	B cannot be celebrity
Stack phase	Push all, pop two, push survivor
Verification	Candidate row all 0; candidate column all 1 except self
Time	$O(n)$
Space	$O(n)$ for linked-list stack
Interview note	Never skip verification

1.34 Mind Map

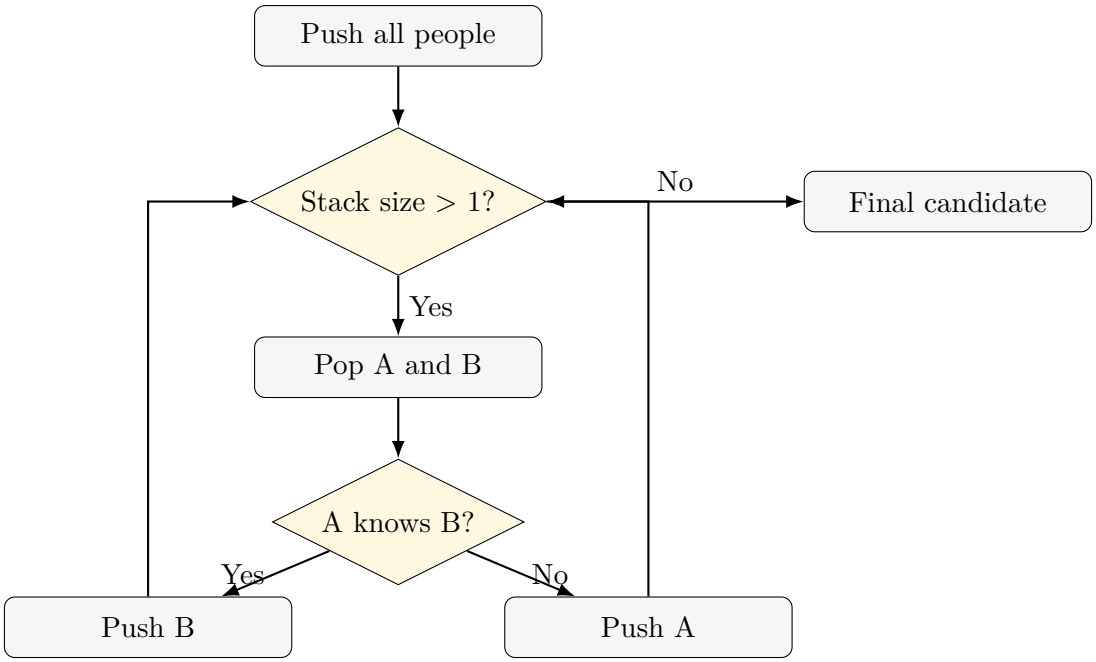
```
Celebrity Problem
|
+-- Definition
|   +-- Everyone knows celebrity
|   +-- Celebrity knows nobody
|
+-- Representation
|   +-- Matrix
|   +-- Directed graph
|   +-- Arrows mean "knows"
|
+-- Observation
|   +-- A knows B: eliminate A
|   +-- A does not know B: eliminate B
|
+-- Stack
|   +-- Push
|   +-- Pop
|   +-- Peek
|   +-- Linked-list top=head
|
+-- Algorithm
|   +-- Eliminate to one candidate
|   +-- Verify candidate
|
+-- Complexity
|   +-- O(n) time
|   +-- O(n) extra space
```

1.35 Flowcharts

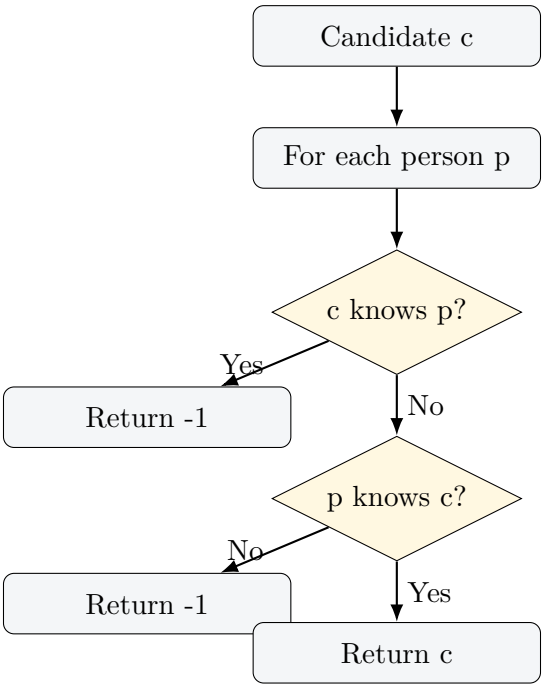
1.35.1 Brute Force Flowchart



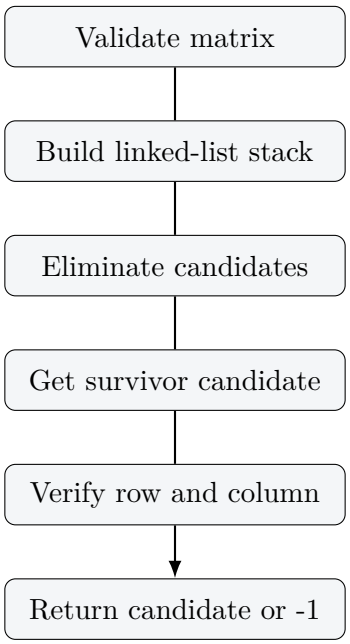
1.35.2 Stack Elimination Flowchart



1.35.3 Verification Flowchart



1.35.4 Overall Algorithm Flowchart



1.36 TikZ Diagrams

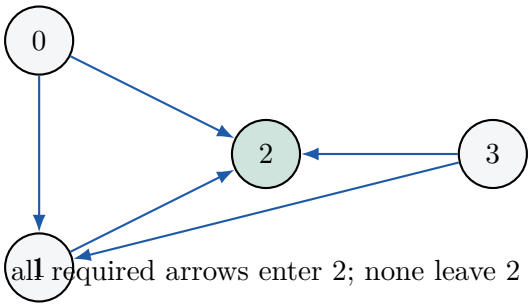
1.36.1 Linked List

1.36.2 Stack

1.36.3 Memory Layout and Pointer Movement

top moves here after push

1.36.4 Relationship Graph



1.37 Practice Questions

1.37.1 Easy

1. What does $M[2][5] = 1$ mean?
2. Define LIFO in your own words.
3. If A knows B, who is eliminated?
4. Why must we verify the final candidate?

1.37.2 Medium

1. Run the stack algorithm on a 5-person matrix of your choice.
2. Modify the code to print every comparison.
3. Write a function that checks only the candidate row.

1.37.3 Hard

1. Implement the $O(1)$-space two-pointer version.
2. Prove that two celebrities cannot exist when $n > 1$.
3. Count the exact maximum number of matrix reads in this implementation.

1.37.4 MCQs

1. If $M[A][B] = 0$, who is eliminated? A) A B) B C) both D) neither.
2. The stack solution's extra space is: A) $O(1)$ B) $O(n)$ C) $O(n^2)$ D) $O(\log n)$.
3. The final survivor is: A) definitely celebrity B) candidate C) invalid D) always -1.

Answers: 1) B, 2) B, 3) B.

1.38 Debugging Exercises

1.38.1 Bug 1: Missing Verification

```
def find_celebrity_bug(matrix):
    # elimination happens here
    return candidate
```

Problem. The survivor may not satisfy celebrity rules. **Fix.** Call `verify_candidate` before returning.

1.38.2 Bug 2: Wrong Direction

```
if knows(matrix, person_b, person_a):
    stack.push(person_b)
else:
    stack.push(person_a)
```

Problem. The code asks the opposite question from the reasoning. **Fix.** Use `knows(matrix, person_a, person_b)` and apply the correct elimination rule.

1.38.3 Bug 3: Broken Push

```
def push(self, value):
    new_node = Node(value)
    self.top = new_node
    new_node.next = self.top
```

Problem. The node points to itself and the old stack is lost. **Fix.** Set `new_node.next = self.top` before `self.top = new_node`.

1.39 Coding Exercises

1. **Easy:** Add a `display()` method that prints the stack from top to bottom.
2. **Easy:** Write tests for one-person and no-celebrity inputs.
3. **Intermediate:** Return both the celebrity and a list of eliminated people.
4. **Intermediate:** Add tracing output controlled by a Boolean flag.
5. **Advanced:** Implement the two-pointer version and compare matrix reads.
6. **Challenge:** Build a random party generator and estimate how often a celebrity appears.

1.40 Mini Projects

1. **Party management simulator:** Create people, define who knows whom, and run the finder.
2. **Celebrity finder visualization:** Animate stack states after every comparison.
3. **Interactive stack simulator:** Let users push, pop, peek, and view linked-list memory.
4. **Performance comparison tool:** Compare brute force, stack, and two-pointer approaches.

1.41 Reflection Questions

1. Why is a candidate not the same as a confirmed answer?
2. What information does one comparison reveal?
3. Why does the linked-list head make a good stack top?
4. When would you prefer a two-pointer solution over a stack solution?
5. How would you explain this algorithm to a non-programmer?

1.42 Revision Notes

- A celebrity has in-degree $n-1$ and out-degree 0 in the group graph.
- $M[i][j] = 1$ means i knows j.
- If A knows B, eliminate A.
- If A does not know B, eliminate B.
- Stack elimination leaves one candidate after $n-1$ comparisons.
- Verification is mandatory.
- Linked-list push and pop at head are $O(1)$.
- Stack method: $O(n)$ time, $O(n)$ extra space.

1.43 Glossary

Term	Definition
Algorithm	A step-by-step method for solving a problem.
Array	A sequence of items stored in indexed positions.
Brute force	A direct solution that tries many or all possibilities.
Candidate	A possible answer not yet fully verified.
Celebrity	A person known by everyone else who knows nobody else.
Complexity	A description of how resource usage grows with input size.
Directed graph	A set of nodes connected by arrows with direction.
Edge case	An unusual input that may reveal bugs.
Elimination	Removing an impossible answer from consideration.
Head	The first node in a linked list.
Heap	The memory area where objects such as nodes are stored.
Index	A numeric position, often starting at 0.
LIFO	Last In, First Out; the stack access rule.
Linked list	Nodes connected by references.
Matrix	A rectangular table of values arranged in rows and columns.
Node	An object holding data and a reference to another node.
Pointer/reference	A value that identifies another object in memory.
Pop	Remove and return the top stack item.
Push	Add an item to the top of a stack.
Space complexity	How extra memory grows with input size.
Stack	A LIFO data structure.
Time complexity	How running time grows with input size.
Verification	Final checking that a candidate satisfies all requirements.